

4.3 Command Line

Sue has just finished writing a program for her mother to convert columns of Euros to Dollars. In order to make this program as convenient and user-friendly as possible, she chooses to allow the input to be specified by command line parameters.

Objectives

By the end of this chapter, you will be able to:

- Write a program to accept passed parameters.
- Understand jagged arrays.

Prerequisites

Before reading this chapter, please make sure you are able to:

- Create a function in C++ (Chapter 1.4).
- Pass data into a function using both pass-by-value and pass-by-reference (Chapter 1.4).
- Declare a pointer variable (Chapter 3.3).
- Pass a pointer to a function (Chapter 3.3).
- Declare a multi-dimensional array (Chapter 4.0).

Overview

Most operating systems (Windows, Unix variants such as Linux and OS X, and others) enable you to pass parameters to your program when the program starts. In other words, it is possible for the user to send data to the program before the program is run. Most of the Linux commands you have been using all semester take command line parameters:

Command	Example	Parameters
List the contents of a directory	<code>ls *.cpp</code>	<code>*.cpp</code>
Submit a homework assignment	<code>submit project4.cpp</code>	<code>project4.cpp</code>
Change to a new directory	<code>cd submittedHomework</code>	<code>submittedHomework</code>
Copy a file from one location to another	<code>cp /home/cs124/template.cpp assignment43.cpp</code>	<code>/home/cs124/template.cpp</code> <code>assignment43.cpp</code>

In each of these cases, the programmers configured their programs to accept command line parameters. The purpose of this chapter is to learn how to do this for our programs.

Syntax

To configure your program to accept command line parameters, it is necessary to define `main()` a little differently than we have done this semester up to this point.

```
int main(int argc, char ** argv)
{
    ...
}
```

Observe the two cryptic parameter names. In the past we left the parameter list of `main()` empty. When this list is empty, we are ignoring any command line parameters the user may send to us. However, when we specify the `argc` and `argv` parameters, we can access the user-sent data from within our program.

argc

The first parameter is the number of arguments or parameters the user typed. For example, if the program name is `a.out` and the user typed...

```
a.out one two three
```

... then `argc` would equal 4 because four arguments were typed: `a.out`, `one`, `two`, `three`. Just like with any passed parameter, the user can name this anything he wants. The convention is to use the name `argc`, but you may want to name it something more meaningful like `countArgument`.

argv

The second parameter is the list of arguments. In all cases, we get an array of c-strings. Back to our example above where the user typed...

```
a.out one two three
```

... then `argv[0]` equals `"a.out"` and `argv[1]` equals `"one"`. Again, note that the first parameter is always the name of the program. As with `argc`, `argv` is not the best name. It means “argument vector” or “list of unknown arguments.” Possibly a better name would be `listArguments`.

Note the unusual `**` notation for the syntax of `argv`. This means that we have a pointer to a pointer. Possibly a better definition would be:

```
int main(int argc, char * argv[])
{
    ...
}
```

This might better illustrate what is going on. Here, we have an array of c-strings. How many items are in the c-string? This is where `argc` comes into play.

Example 4.3 – Reflect the Command Line	
Demo	This example will demonstrate how to accept data passed from the user through the command line.
Problem	Write a program to display on the screen the name of the program, how many parameters were passed through the command line, and the contents of each parameter. <pre> /home/cs124/examples> <u>a.out one two three</u> The name of the program is: a.out There are 3 parameters Parameter 1: one Parameter 2: two Parameter 3: three </pre>
Solution	The first thing to look for is how <code>main()</code> has two parameters. Of course we can call these anything we want, but <code>argc</code> and <code>argv</code> are the standard names. Next, observe how <code>argv[0]</code> is the name of the program. Finally, we access each parameter with <code>argv[iArg]</code>. <pre> #include <iostream> using namespace std; /***** * MAIN: Reflect back to the user what he * typed on the command prompt *****/ int main(int argc, char ** argv) // again, MAIN really takes two parameters { // name of the program cout << "The name of the program is: " << argv[0] // the first item in the list is always << endl; // the command the user typed // number of parameters cout << "There are " << argc - 1 // don't forget to subtract one due to << " parameters\n"; // the first being the program name // show each parameter for (int iArg = 1; iArg < argc; iArg++) // standard command line loop cout << "\tParameter" << iArg << ": " << argv[iArg] << endl; // each argv[i] is a c-string return 0; } </pre>
Challenge	As a challenge, try to rename <code>a.out</code> (using the <code>mv</code> command) and run it again. How do you suppose the program knows that it was renamed after compilation?
See Also	The complete solution is available at 4-3-commandLine.cpp or: <pre>/home/cs124/examples/4-3-commandLine.cpp</pre> 

Example 4.3 – Get Filename

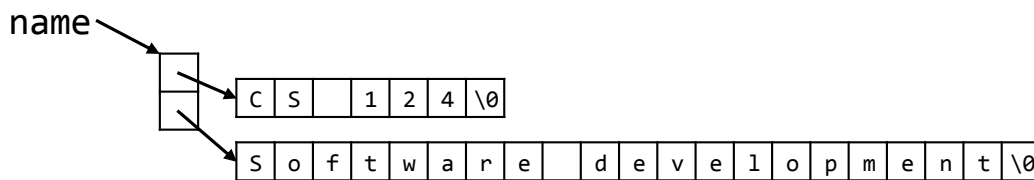
Demo	This example will demonstrate one of the most common uses for command line parameters: fetching the filename from the user. This will be done using the string class.	
Problem	Write a program to prompt the user for a filename. This can be provided either through a command line parameter or, if none was specified, through a traditional prompt. <pre data-bbox="277 390 821 453">/home/cs124/examples> a.out fileName.txt The filename is "fileName.txt"</pre> <pre data-bbox="277 495 821 579">/home/cs124/examples> a.out Please enter the filename: fileName.txt The filename is "fileName.txt"</pre>	
Solution	There are three parts to this program. First, the program will determine if an erroneous number of parameters were specified. If this proves to be the case, the program exists with a suitable error message. Next, the program prompts the user for a filename if there are no parameters specified on the command line. Finally, if there is a parameter specified, it is copied into the <code>fileName</code> variable. <pre data-bbox="277 789 1386 1419">int main(int argc, char ** argv) { // ensure the correct number of parameters was specified if (argc > 2) // one for the name of the program, one for the filename { cout << "Unexpected number of parameters.\nUsage:\n"; cout << "\t" << argv[0] << " [filename]\n"; return 1; } // parse the command line string fileName; if (argc == 1) // only the program name was specified fileName = getFilename(); else fileName = argv[1]; // display the results cout << "The filename is \"" << fileName << "\"\n"; return 0; }</pre>	
Challenge	As a challenge, can you do this without the <code>string</code> class? Hint: use <code>strcpy()</code> from the <code>cstring</code> library. Another challenge is to modify Project 3 to accept a filename as a command line parameter. Can you do this for Project 4 as well?	
See Also	The complete solution is available at 4-3-getFilename.cpp or: <pre data-bbox="277 1675 821 1707">/home/cs124/examples/4-3-getFilename.cpp</pre> 	

Jagged Arrays

Multi-dimensional arrays can be thought of as arrays of arrays where each row is the same length and each column is the same length. In other words, multi-dimensional arrays are square. There is another form of “arrays of arrays” where each row is not the same length. We call these “jagged arrays.” Consider the following code:

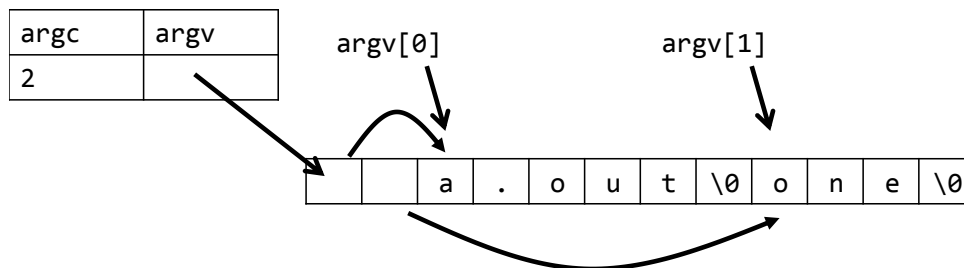
```
{
    char * name[] =
    {
        "CS 124",
        "Software development"
    };
}
```

The first string is a different size than the second. Thus we have:



In this example, we have declared an array of pointers to characters (`char *`). Each one of these pointers is pointing to a string literal. Observe how each string is of a different size. Also, the two strings do not need to be next to each other in memory as multi-dimensional arrays do.

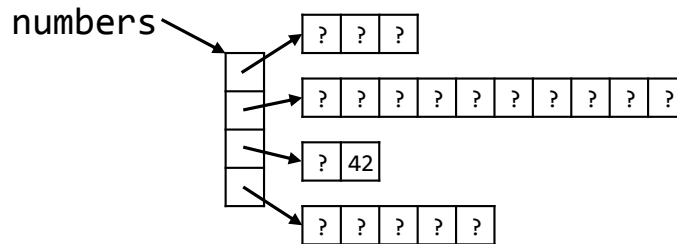
It turns out that `argv` works much like a pointer to dynamically allocated memory. The only difference is that the operating system typically puts each command line parameter immediately after the array of pointers in memory:



Example 4.3 – Jagged Array of Numbers

Demo

This example will demonstrate how to create a jagged array of integers. In this case, each row will be allocated dynamically. There will be four rows, each containing a different number of items.



Solution

There are five steps to setting up an jagged array of numbers:

1. Allocate the array of pointers. This must happen before the rows are allocated.
2. Allocate each row individually. This will require separate new statements.
3. Use the array. In this case, the value '42' will be assigned to one cell.
4. Free the individual rows.
5. Free the array of pointers. This must happen after the individual rows are freed.

Note that if the number of rows is known at compile time then step 1 and 5 can be skipped.

```
{
    int ** numbers;                // pointer to a pointer to an integer

    // 1. allocate the array of arrays.
    numbers = new int *[4];        // an array of pointers to integers

    // 2. allocate each individual array
    numbers[0] = new int[3];       // an array of integers
    numbers[1] = new int[10];
    numbers[2] = new int[2];
    numbers[3] = new int[5];

    // 3. assign a value
    numbers[2][1] = 42;           // access is the same as with standard
                                // multi-dimensional array

    // 4. free each array
    for (int i = 0; i < 4; i++)    // we must free each individual array or we
        delete [] numbers[i];     // will forget about them and have a leak

    // 5. free the array of arrays
    delete [] numbers;           // finally free the original array
}
```

Challenge

As a challenge, change the code to work with arrays of floats. Make it work with five rows instead of four (making sure to free each row when finished). Finally, try to fill each item in the array with a number.

See Also

The complete solution is available at [4-3-jaggedArrayNumber.cpp](#) or:

/home/cs124/examples/4-3-jaggedArrayNumber.cpp



Example 4.3 – Jagged Array of Strings

Demo

This example will demonstrate how to create a jagged array of strings. This allows us to use exactly the amount of memory required to represent the user's data.

Problem

Write a program to prompt the user for his name and store his name in a jagged array.

How man letters are there in your first, middle, and last name?

4 5 20

Please enter your first name: John

Please enter your middle name: Jacob

Please enter your last name: Jingleheimer Schmidt

Your name is "John Jacob Jingleheimer Schmidt"

Solution

To allocate the jagged array, we need to know the size of each row (sizeFirst, sizeMiddle, sizeLast).

```
char ** names;  
names = new char *[3]; // three names  
names[0] = new char[sizeFirst + 1];  
names[1] = new char[sizeMiddle + 1];  
names[2] = new char[sizeLast + 1];
```

From here, it behaves like any other multi-dimensional array.

```
// fill the jagged array  
cout << "Please enter your first name: ";  
cin.getline(names[0], sizeFirst + 1);  
cout << "Please enter you middle name: ";  
cin.getline(names[1], sizeMiddle + 1);  
cout << "Please enter your last name: ";  
cin.getline(names[2], sizeLast + 1);  
  
// display the results  
cout << "Your name is \"  
    << names[0] << ' '  
    << names[1] << ' '  
    << names[2] << "\"\\n\"";
```

Finally, it is important to free the rows and the pointers in the correct order.

```
delete [] names[0];  
delete [] names[1];  
delete [] names[2];  
delete [] names;
```

Challenge

As a challenge, can you add another row to correspond to the user's title ("Dr.")? Again, don't forget to allocate the row and free the row.

See Also

The complete solution is available at [4-3-jaggedArrayString.cpp](#) or:

/home/cs124/examples/4-3-jaggedArrayString.cpp



Problem 1

Given the following program:

```
int main(int argc, char ** argv)
{
    cout << argv[0] << endl;
    return 0;
}
```

What is the output if the user runs the following command:

%a.out one two three

Answer:

Please see page 348 for a hint.

Problem 2

Given the following program:

```
int main(int argc, char ** argv)
{
    <code goes here>
}
```

Write the code that will display the value “three”:

%a.out one two three
three

Answer:

Please see page 348 for a hint.

Problem 3

Given the following program:

```
int main(int argc, char ** argv)
{
    cout << argc << endl;

    return 0;
}
```

What is the output if the user runs the following command:

%a.out one two

Answer:

Please see page 348 for a hint.

Problem 4

Given the following program:

```
int main(int argc, char ** argv)
{
    cout << atoi(argv[1])
          + atoi(argv[2]);
    return 0;
}
```

What is the output if the user types in:

a.out 4 5 6

Answer:

Please see page 348 for a hint.

Problem 5

Describe, in English, the functionality of the following program:

```
int main(int argc, char ** argv)
{
    while (--argc)
        cout << argv[argc] << endl;
    return 0;
}
```

Answer:

Please see page 347 for a hint.

Problem 6

Write a program to compute the absolute value given numbers specified on the command line:

```
%a.out -4.2 96.2 -3.90
The absolute value of -4.2 is 4.2
The absolute value of 96.2 is 96.2
The absolute value of -3.90 is 3.9
```

Answer:

Please see page 347 for a hint.

Assignment 4.3

Write a program to convert feet to meters. The conversion from feet to meters is:

1 foot = 0.3048 meters

The input will be numbers passed on the command line. You will want to use the library function `atof` to convert a string into a float. For example, consider the following code:

```
#include <cstdlib>    // the library for atof()
#include <iostream>
using namespace std;

int main()
{
    char text[] = "3.14159";    // a c-string
    float pi;                  // the float where the answer will go
    pi = atof(text);            // atof() translates the c-string into a float
    cout << pi << endl;        // this better be 3.14159

    return 0;
}
```

Pay special attention to the `#include <cstdlib>` code; you will need that for this assignment.

Example

Consider the output of a program called `a.out`:

```
a.out 1 2 3
1.0 feet is 0.3 meters
2.0 feet is 0.6 meters
3.0 feet is 0.9 meters
```

Assignment

The test bed is available at:

```
testBed cs124/assign43 assignment43.cpp
```

Don't forget to submit your assignment with the name "Assignment 43" in the header.

Please see page 348 for a hint.